Raport z etapu 9

Przetwarzanie dużych zbiorów danych

Szymon Jędrzejczak (266867)
Mateusz Nasewicz (268513)
Szymon Wieczorek (266586)

03.06.2026

Spis treści

1 Przegląd implementacji etapów PySpark **2**

2 Wybrane fragmenty kodu źródłowego **2**
2.1 E1.1 – Złączenie z Broadcast (Map-Side Join w Sparku) 2
2.2 E2 – Reduce-Side Join na RDD z obliczeniem wskaźników 3
2.3 E3 – Korelacja Pearsona przez Spark SQL 3
2.4 Tryb holistyczny (całość potoku w jednym grafie) 4

3 Czasy wykonania potoku PySpark **5**

4 Eksperymenty i optymalizacja **5**
4.1 Eksperyment A: Broadcast Join vs. Sort-Merge Join (E1.1) 5
4.2 Eksperyment B: Liczba partycji shuffle w E1.5 6

5 Logi z wykonania zadań Spark **6**

1 Przegląd implementacji etapów PySpark

System został przeportowany z paradygmatu MapReduce (Hadoop) na platformę Apache Spark przy użyciu interfejsu PySpark. Pełny potok przetwarzania danych zawiera te same etapy analityczne co w etapie 7, jednak zrealizowane z wykorzystaniem transformacji na rozproszonych zbiorach danych (RDD) oraz DataFrame API. Potok uruchamiany jest w trybie `-master yarn -deploy-mode client` na tym samym klastrze Docker co poprzednio.

- **E1.1 (Ekstrakcja i normalizacja GTFS)** – wczytuje pliki `stop_times.txt`, `trips.txt`, `routes.txt`, `calendar.txt` oraz `stops.txt` z HDFS jako DataFrame. Filtruje kursy dla dni roboczych, a następnie wykonuje złączenia z opcjonalnym użyciem `broadcast()` dla małych tabel (Map-Side Join). Wynik zapisywany jest w formacie Parquet.
- **E1.5 (Pre-agregacja GTFS)** – wczytuje wynik E1.1, filtruje odjazdy w godzinach szczytu (7–9 rano) i agreguje liczbę odjazdów per przystanek. Parametr `-reducers` steruje liczbą partycji shuffle (`spark.sql.shuffle.partitions`).
- **E1.2 (Ekstrakcja OSM)** – wczytuje znormalizowany CSV z POI z HDFS, odfiltrowuje rekordy z zerową lokalizacją, nadaje każdemu obiektowi klucz Geohash (precyzja 6) i zapisuje wynik jako Parquet.
- **E1.3 (Normalizacja ofert nieruchomości)** – wczytuje geokodowane pliki CSV (separator `;`) z HDFS, rzutuje kolumny cenowe i geometryczne oraz oblicza Geohash per oferta. Wynik w formacie Parquet.
- **E2 (Obliczanie wskaźników per nieruchomość)** – scala trzy zbiory (przystanki, POI, oferty) po kluczu Geohash metodą `groupByKey` na RDD. Dla każdej oferty oblicza wskaźniki dostępności komunikacyjnej, handlu, gastronomii, rekreacji, edukacji i hałasu na podstawie odległości Haversine do obiektów w promieniu 800 m. Wynik zapisywany jest jako plik tekstowy kompatybilny z formatem wyjściowym E2Reducer z etapu 7.
- **E3 (Macierz korelacji Pearsona)** – wczytuje wyjście E2, parsuje linie do DataFrame, grupuje per miasto i oblicza korelacje Pearsona między sześcioma wskaźnikami a trzema zmiennymi cenowymi (`cenaM2`, `cenaCalkowita`, `powierzchnia`) przy użyciu funkcji `corr()`.

2 Wybrane fragmenty kodu źródłowego

2.1 E1.1 – Złączenie z Broadcast (Map-Side Join w Sparku)

Poniższy fragment przedstawia realizację złączenia Map-Side w PySpark. Małe tabele (trasy, kalendarz, przystanki) owijane są funkcją `broadcast()`, co eliminuje shuffle i odpowiada semantycznie złączeniu po stronie mappera z etapu MapReduce. Flaga `-disable_broadcast` pozwala wyłączyć tę optymalizację na potrzeby eksperymentu A.

```
def _logic_e1_1(spark, stop_times_path, cache_dir, disable_broadcast):
    stop_times = spark.read.csv(stop_times_path, header=True)
    trips = spark.read.csv(f"{cache_dir}/trips.txt", header=True)
    routes = spark.read.csv(f"{cache_dir}/routes.txt", header=True)
    calendar = spark.read.csv(f"{cache_dir}/calendar.txt", header=True)
    stops = spark.read.csv(f"{cache_dir}/stops.txt", header=True)

    calendar = calendar.filter((col("monday") == "1") | (col("tuesday") == "1") |
                               (col("wednesday") == "1") | (col("thursday") == "1") |
                               (col("friday") == "1")).select("service_id").distinct()

    routes = routes.select("route_id", "route_type")
    trips = trips.select("trip_id", "route_id", "service_id")

    def b(df):
        return df if disable_broadcast else broadcast(df)

    trips_info = trips.join(b(routes), "route_id").join(b(calendar), "service_id")

    stops = stops.select("stop_id", col("stop_lat").cast("double"),
    ↪  col("stop_lon").cast("double"))
    geohash_udf = udf(lambda lat, lon: geohash_encode(lat, lon, 6), StringType())
    stops = stops.withColumn("geoId", geohash_udf("stop_lat", "stop_lon"))
```

```
23
24      stop_times = stop_times.select("trip_id", "arrival_time", "stop_id")
25
26      joined = stop_times.join(b(trips_info), "trip_id") \
27                         .join(b(stops), "stop_id")
28
29      res = joined.select("stop_id", "stop_lat", "stop_lon", "geoId", "route_type",
        ↪  "arrival_time")
30      return res
31
32  def run_e1_1(spark, stop_times_path, cache_dir, output_path, disable_broadcast=False):
33      print("==> Running E1.1: GTFS Extract + Map Side Join")
34      res = _logic_e1_1(spark, stop_times_path, cache_dir, disable_broadcast)
35      res.write.parquet(output_path, mode="overwrite")
```

2.2 E2 – Reduce-Side Join na RDD z obliczeniem wskaźników

Etap E2 łączy trzy strumienie danych po kluczu Geohash metodą `union` + `groupByKey` na RDD, co odpowiada Reduce-Side Join z etapu 7. Dla każdej oferty obliczane są wskaźniki na podstawie algorytmu Haversine od przystanków i punktów POI w promieniu 800 m.

```
1   def _logic_e2(stops_df, pois_df, offers_df):
2       stops_rdd = stops_df.rdd.map(lambda r: (r.geoId, ('stop', r.asDict())))
3       pois_rdd = pois_df.rdd.map(lambda r: (r.geoId, ('poi', r.asDict())))
4       offers_rdd = offers_df.rdd.map(lambda r: (r.geoId, ('offer', r.asDict())))
5
6       joined_rdd = stops_rdd.union(pois_rdd).union(offers_rdd)
7
8       res_rdd = joined_rdd.groupByKey().mapPartitions(lambda it: (out for key, vals in it for out
        ↪  in compute_indicators(key, [(key, v) for v in vals])))
9       return res_rdd
10
11  def run_e2(spark, t1_5_dir, t1_2_dir, t1_3_dir, output_dir):
12      print("==> Running E2: Join & Indicators calculation")
13      stops_df = spark.read.parquet(t1_5_dir)
14      pois_df = spark.read.parquet(t1_2_dir)
15      offers_df = spark.read.parquet(t1_3_dir)
16
17      res_rdd = _logic_e2(stops_df, pois_df, offers_df)
18      res_rdd.saveAsTextFile(output_dir)
```

2.3 E3 – Korelacja Pearsona przez Spark SQL

W odróżnieniu od ręcznej implementacji wzoru Pearsona w Java (etap 7), wersja PySpark korzysta z wbudowanej funkcji `corr()` z modułu `pyspark.sql.functions`, co upraszcza kod i deleguje obliczenia do zoptymalizowanego silnika Catalyst.

```
1   def _logic_e3(rdd):
2       def parse_line(line):
3           def f(s):
4               try: return float(s)
5               except: return 0.0
6           csv_part = line.split('\t')[1]
7           parts = csv_part.split(',')
8           return {
9               'miasto': parts[5],
10              'cenaM2': f(parts[2]),
11              'cenaCalkowita': f(parts[3]),
12              'powierzchnia': f(parts[4]),
13              'wskPrzep': f(parts[6]),
14              'indHandlu': f(parts[7]),
15              'indGastro': f(parts[8]),
16              'indRekr': f(parts[9]),
```

```
            'indEduk': f(parts[10]),
            'wspHalas': f(parts[11])
        }

    df = rdd.map(parse_line).toDF()

    indicators = ['wskPrzep', 'indHandlu', 'indGastro', 'indRekr', 'indEduk', 'wspHalas']
    prices = ['cenaM2', 'cenaCalkowita', 'powierzchnia']

    aggs = []
    for x in indicators:
        for y in prices:
            aggs.append(corr(x, y).alias(f"{x}_{y}"))

    res_df = df.groupBy("miasto").agg(*aggs)

    def format_output(row):
        out = []
        mapping = {
            'wskPrzep': 'wsk_przepustowosci', 'indHandlu': 'ind_handlu',
            'indGastro': 'ind_gastro', 'indRekr': 'ind_rekreacji',
            'indEduk': 'ind_edukacji', 'wspHalas': 'wsp_halas'
        }
        for ind in indicators:
            c_m2 = row[f"{ind}_cenaM2"] or 0.0
            c_cal = row[f"{ind}_cenaCalkowita"] or 0.0
            c_pow = row[f"{ind}_powierzchnia"] or 0.0

            fmt = lambda v: f"{v:.6f}" if not math.isnan(v) else "0.000000"
            line = f"{row['miasto']}\t{mapping[ind]},{fmt(c_m2)},{fmt(c_cal)},{fmt(c_pow)}"
            out.append(line)
        return out

    final_rdd = res_df.rdd.flatMap(format_output)
    return final_rdd

def run_e3(spark, input_dir, output_dir):
    print("==> Running E3: Pearson Correlation Matrix")
    rdd = spark.sparkContext.textFile(input_dir)
    final_rdd = _logic_e3(rdd)
    final_rdd.saveAsTextFile(output_dir)
```

2.4 Tryb holistyczny (całość potoku w jednym grafie)

Poniższy kod przedstawia wykonanie całego potoku w jednym grafie skierowanym (DAG). Poszczególne etapy wywoływane są jako funkcje przekazujące referencje do obiektów DataFrame/RDD, bez zapisu wyników pośrednich do HDFS. Ostatnia funkcja wyzwala wszystkie operacje (leniwa ewaluacja) i zapisuje wynik bezpośrednio do HDFS.

```
def run_prod(spark, args):
    res_e1_1 = _logic_e1_1(spark, args.stop_times, args.gtfs_cache, args.disable_broadcast)
    res_e1_5 = _logic_e1_5(res_e1_1, args.reducers)

    res_e1_2 = _logic_e1_2(spark, args.osm_csv)
    res_e1_3 = _logic_e1_3(spark, args.re_geocoded)

    res_e2_rdd = _logic_e2(res_e1_5, res_e1_2, res_e1_3)
    final_rdd = _logic_e3(res_e2_rdd)

    final_rdd.saveAsTextFile(args.t3)
```

3 Czasy wykonania potoku PySpark

Wyniki uzyskane podczas uruchomienia pełnego potoku PySpark na tym samym klastrze testowym co w etapie 7 zebrano w tabeli 3.1. Czasy zostały uśrednione z 3 pełnych iteracji (pominięto czas przesyłania danych wejściowych do HDFS). Wszystkie etapy wariantu sekwencyjnego operują na danych w formacie Parquet w HDFS.

Tabela 3.1: Czasy wykonania poszczególnych etapów potoku PySpark dla 3 uruchomień oraz czas średni

Etap	Opis operacji	Run 1 [s]	Run 2 [s]	Run 3 [s]	Średni czas [s]
E1.1	Ekstrakcja GTFS + złączenie z broadcast	24	25	23	24.0
E1.5	Agregacja GTFS (odjazdy/przystanek)	19	17	15	17.0
E1.2	Ekstrakcja POI z OSM CSV	18	16	17	17.0
E1.3	Normalizacja ofert nieruchomości	21	21	21	21.0
E2	Obliczanie wskaźników (groupByKey + Haversine)	27	25	25	25.7
E3	Korelacja Pearsona per miasto	19	19	20	19.3
	Łączny czas wykonania (uruchomienia sekwencyjne)	128	123	121	124.0
	Łączny czas wykonania (holistyczny DAG w pamięci RAM)	**39**	**39**	**42**	**40.0**

Pod tabelą zamieszczono dodatkowy pomiar dla uruchomienia potoku w tzw. trybie holistycznym. Zamiast uruchamiać każdy etap jako oddzielne zadanie `spark-submit` i zrzucać dane pośrednie do HDFS na koniec każdego etapu, powiązano wszystkie DataFrame i RDD ze sobą w jeden ciągły graf DAG. W rezultacie Spark Catalyst Optimizer mógł optymalnie wykonać całe zadanie naraz, operując głównie w pamięci RAM, bez overheadu maszyny wirtualnej, obniżając czas do **40 sekund** (przyspieszenie ok. 80% względem bazowego MapReduce).

4 Eksperymenty i optymalizacja

Przeprowadzono dwa eksperymenty badające wpływ konfiguracji Sparka na czas i koszt przetwarzania.

4.1 Eksperyment A: Broadcast Join vs. Sort-Merge Join (E1.1)

W etapie E1.1 małe tabele referencyjne (trasy, kalendarz, przystanki) są domyślnie przekazywane jako zmienne broadcast do wszystkich executorów, co eliminuje fazę shuffle przy złączeniu (*Map-Side Join* w terminologii MapReduce). Eksperyment porównuje ten wariant z wyłączonymi hintami broadcast, w którym Spark stosuje Sort-Merge Join wymagający przetasowania danych.

Tabela 4.1: Wyniki eksperymentu A – Broadcast Join vs. Sort-Merge Join (E1.1)

Wariant	Run 1 [s]	Run 2 [s]	Run 3 [s]	Średni czas [s]
Broadcast włączony	20	20	20	20.0
Broadcast wyłączony	23	21	22	22.0

Wniosek: Tabele referencyjne GTFS (trasy, kalendarz) są na tyle małe, że ich explicit broadcast skutecznie eliminuje fazę shuffle, dając w efekcie średnio 2 sekundy zysku (ok. 9%). Optymalizacja ta dowodzi swojej przydatności przy złączeniach małych tabel wymiarów (Map-Side Join), odciążając dyski i sieć, nawet na niewielkim zbiorze danych.

4.2 Eksperyment B: Liczba partycji shuffle w E1.5

Parametr `spark.sql.shuffle.partitions` bezpośrednio odpowiada liczbie reducerów w modelu MapReduce – określa, na ile partycji zostanie podzielony wynik operacji grupowania. Eksperyment weryfikuje, czy zwiększenie tego parametru przyspiesza agregację GTFS per przystanek.

Tabela 4.2: Wyniki eksperymentu B – Liczba partycji shuffle (E1.5)

Liczba partycji	Run 1 [s]	Run 2 [s]	Run 3 [s]	Średni czas [s]
1	16	15	13	14.7
10	16	18	16	16.7
50	19	16	19	18.0

Wniosek: Zwiększanie liczby partycji shuffle z 1 do 50 w przypadku relatywnie małego zbioru GTFS doprowadziło do odwrotnego od oczekiwanego skutku – wydłużenia czasu wykonania z 14.7s do 18.0s. Wynika to z faktu, że narzut związany z dystrybucją, planowaniem i uruchamianiem kilkudziesięciu malutkich tasków przez framework przerasta zyski ze zrównoleglenia operacji grupowania. W modelu MapReduce szczytowa wydajność osiągnięta została przy 2 reducerach, natomiast silnik Spark najwydajniej zagregował ten wolumen danych w obrębie zaledwie jednej partycji.

5 Logi z wykonania zadań Spark

Poniżej zamieszczono fragment logów konsoli dla etapu **E2** (Obliczanie wskaźników per nieruchomość) z trzeciego uruchomienia potoku. Widoczne jest połączenie z ResourceManagerem YARN, postęp wykonania jobu oraz informacje o liczbie uruchomionych tasków i etapach DAG.

```
[INFO]  ==> START: E2 (00:24:58)
Deleted /user/hadoop/t2_py
26/06/02 07:25:01 INFO SparkContext: Running Spark version 3.4.0
26/06/02 07:25:01 WARN NativeCodeLoader: Unable to load native-hadoop library...
26/06/02 07:25:01 INFO ResourceUtils: ==============================================
26/06/02 07:25:01 INFO ResourceUtils: No custom resources configured for spark.driver.
26/06/02 07:25:01 INFO ResourceUtils: ==============================================
26/06/02 07:25:01 INFO SparkContext: Submitted application: PySpark Pipeline: e2
26/06/02 07:25:01 INFO ResourceProfile: Default ResourceProfile created...
26/06/02 07:25:01 INFO SecurityManager: Changing view acls to: root
26/06/02 07:25:01 INFO SecurityManager: Changing modify acls to: root
26/06/02 07:25:01 INFO SecurityManager: SecurityManager: authentication disabled; ui acls
↪  disabled;
26/06/02 07:25:01 INFO Client: Submitting application application_1780348845350_0096 to
↪  ResourceManager
26/06/02 07:25:03 INFO YarnClientImpl: Submitted application application_1780348845350_0096
26/06/02 07:25:04 INFO Client: Application report for application_1780348845350_0096 (state:
↪  ACCEPTED)
26/06/02 07:25:06 INFO Client: Application report for application_1780348845350_0096 (state:
↪  RUNNING)
26/06/02 07:25:08 INFO YarnClientSchedulerBackend: SchedulerBackend is ready for scheduling
↪  beginning after reached minRegisteredResourcesRatio: 0.8
==> Running E2: Join & Indicators calculation
26/06/02 07:25:08 INFO SharedState: Setting hive.metastore.warehouse.dir...
26/06/02 07:25:09 INFO DAGScheduler: Submitting ResultStage 0 (MapPartitionsRDD[1]...)
26/06/02 07:25:09 INFO YarnScheduler: Adding task set 0.0 with 1 tasks...
26/06/02 07:25:10 INFO TaskSetManager: Finished task 0.0 in stage 0.0 (TID 0) in 769 ms on
↪  slave3
...
26/06/02 07:25:12 INFO SparkContext: Starting job: runJob at SparkHadoopWriter.scala:83
26/06/02 07:25:12 INFO DAGScheduler: Submitting ShuffleMapStage 3 (PairwiseRDD[30] at
↪  groupByKey...)
26/06/02 07:25:21 INFO DAGScheduler: ShuffleMapStage 3 (groupByKey...) finished in 8.538 s
```

```
...
26/06/02 07:25:35 INFO DAGScheduler: Job 3 finished: runJob at SparkHadoopWriter.scala:83, took
↪  24.835 s
26/06/02 07:25:35 INFO YarnScheduler: Killing all running tasks in stage 4: Stage finished
26/06/02 07:25:35 INFO DAGScheduler: Job 3 finished: runJob at SparkHadoopWriter.scala:83, took
↪  24.835118 s
26/06/02 07:25:35 INFO SparkHadoopWriter: Start to commit write Job
↪  job_202606020725122056370383947161265_0035.
26/06/02 07:25:35 INFO SparkHadoopWriter: Write Job job_202606020725122056370383947161265_0035
↪  committed. Elapsed time: 28 ms.
26/06/02 07:25:35 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/06/02 07:25:35 INFO SparkUI: Stopped Spark web UI at http://spark:4040
26/06/02 07:25:35 INFO YarnClientSchedulerBackend: Shutting down all executors
26/06/02 07:25:35 INFO YarnSchedulerBackend$YarnDriverEndpoint: Asking each executor to shut
↪  down
26/06/02 07:25:35 INFO YarnClientSchedulerBackend: YARN client scheduler backend Stopped
26/06/02 07:25:35 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/06/02 07:25:35 INFO MemoryStore: MemoryStore cleared
26/06/02 07:25:35 INFO BlockManager: BlockManager stopped
26/06/02 07:25:35 INFO BlockManagerMaster: BlockManagerMaster stopped
26/06/02 07:25:35 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint:
↪  OutputCommitCoordinator stopped!
26/06/02 07:25:35 INFO SparkContext: Successfully stopped SparkContext
26/06/02 07:25:36 INFO ShutdownHookManager: Shutdown hook called
26/06/02 07:25:36 INFO ShutdownHookManager: Deleting directory
↪  /tmp/spark-d273f710-120a-43a8-a0bb-e3149f43e13b
26/06/02 07:25:36 INFO ShutdownHookManager: Deleting directory
↪  /tmp/spark-d92d0453-76e8-4cd7-902e-4947530bff0c
26/06/02 07:25:36 INFO ShutdownHookManager: Deleting directory
↪  /tmp/spark-d92d0453-76e8-4cd7-902e-4947530bff0c/pyspark-b0a1f64e-ae54-47f3-b5c9-b5c3e325411b
[TIMING] E2: 38s
```